

Reporte técnico

Materia: Ingeniería de Software I

Nombre del docente: ALEXIA ISABEL CARRAZCO SANCHEZ

Universidad de Sonora

Ingeniería en Sistemas de Información

Elaborado por:

Rodriguez Andrade Denise Abigail

Sánchez Valenzuela Natalia Guadalupe

Hermosillo, Sonora **15 de septiembre, 2026**

Metodología en Cascada

1. Introducción

La metodología en cascada es uno de los enfoques más conocidos dentro de la ingeniería de software. Se caracteriza por organizar el desarrollo de un sistema mediante fases secuenciales, en las que cada etapa produce información y entregables que sirven como base para la siguiente. Antes de iniciar un proyecto se busca definir con detalle qué necesita el cliente, cuáles serán las características del sistema, cómo se construirá, cómo se comprobará su funcionamiento y cómo será entregado y mantenido. Este enfoque reduce la incertidumbre mediante planificación, documentación y controles formales.

Un aspecto clave es la relación entre el momento en que se detecta un problema y el esfuerzo necesario para corregirlo. Un error encontrado en la etapa de requisitos puede solucionarse fácilmente modificando una especificación, mientras que si se descubre en fases posteriores puede implicar cambios en diseño, código, pruebas y documentación. Por ello, la cascada pone especial énfasis en la definición temprana de requisitos y en la aprobación formal de cada etapa.

2. Origen histórico

La organización del desarrollo en etapas existía antes de que se popularizara el término “Waterfall”. En 1956, Herbert D. Benington presentó *Production of Large Computer Programs*, aplicado al sistema SAGE de defensa aérea, donde describió un proceso dividido en planificación, especificaciones, codificación, pruebas y evaluación. Este antecedente muestra que los proyectos grandes ya requerían planificación y coordinación.

En 1970, Winston W. Royce publicó *Managing the Development of Large Software Systems*. Aunque su trabajo suele asociarse al origen del modelo en cascada, Royce no recomendaba un proceso rígido y lineal, sino que advertía sobre sus riesgos y proponía retroalimentación, iteración y prototipos. Con el tiempo, el enfoque secuencial fue adoptado por organizaciones gubernamentales e industriales, consolidándose bajo el nombre de “cascada”.

3. Principios fundamentales

La metodología en cascada se basa en la planificación anticipada, el desarrollo por fases claramente identificadas, la documentación detallada de decisiones, la aprobación formal de cada etapa, la trazabilidad de requisitos y el control de cambios. Estos principios permiten mantener orden y previsibilidad en proyectos donde los requisitos son estables.

4. Fases del ciclo de vida

El ciclo de vida se compone de seis fases principales: análisis de requisitos, diseño del sistema y arquitectura, implementación, pruebas, verificación y despliegue, y mantenimiento. Cada fase tiene objetivos y entregables específicos. El análisis de requisitos documenta las necesidades del cliente en un SRS; el diseño establece arquitectura, módulos y diagramas; la implementación convierte el diseño en código funcional; las pruebas verifican que el sistema cumpla lo establecido; la aceptación y despliegue validan el producto con el cliente; y el mantenimiento asegura correcciones, actualizaciones y adaptaciones para mantener el sistema útil en el tiempo.

5. Documentación y roles

La documentación es un rasgo distintivo de este modelo, ya que cada fase genera entregables como listas de requisitos, diagramas, código fuente, planes de prueba y manuales de usuario. Esto garantiza la conservación del conocimiento del proyecto y facilita auditorías y trazabilidad.

En cuanto a los roles, un proyecto en cascada puede involucrar al cliente o stakeholder, al analista de requisitos, al gerente de proyecto, al arquitecto de software, al equipo de desarrollo y al equipo de pruebas. La distribución depende del tamaño y complejidad del proyecto, aunque en la industria se recomienda mantener una proporción de un QA por cada tres desarrolladores para evitar cuellos de botella.

6. Ventajas y desventajas

Entre las ventajas destacan la claridad en las fases, la documentación extensa, la trazabilidad de requisitos, la planificación anticipada y el control mediante revisiones formales. Estas características la hacen adecuada en entornos regulados o proyectos donde los requisitos son estables.

Sin embargo, también presenta desventajas como la rigidez ante cambios, la retroalimentación tardía, el riesgo de acumulación de errores, la entrega tardía de valor y el alto impacto de modificaciones en etapas avanzadas. Esto se relaciona con la curva del costo del cambio, estudiada por Barry Boehm, que demuestra cómo el costo de corregir errores aumenta significativamente conforme avanza el proyecto. Un requisito incorrecto puede corregirse cambiando un documento, pero un error descubierto en producción puede implicar costos de despliegue, soporte, pérdida de información y afectación a usuarios.

7. El problema del “Big Bang”

Un riesgo frecuente en cascada es que el cliente reciba el producto completo hasta etapas avanzadas. Esto puede generar la situación conocida como “Big Bang”, donde una gran cantidad de funcionalidades llegan juntas al final y el cliente descubre que alguna característica no era lo esperado, que falta una función o que una decisión inicial ya no corresponde a sus necesidades actuales. Mientras más tarde se descubra el problema, mayor será el trabajo necesario para corregirlo.

8. Ejemplo práctico

Un caso ilustrativo es la aplicación móvil “Rápido y Seguro”. Durante las fases iniciales se definieron requisitos como registro de usuarios, catálogo y pagos en línea. Sin embargo, en las pruebas finales el cliente solicitó una función de rastreo GPS en tiempo real. Aunque parecía un cambio menor, su incorporación implicó modificar requisitos, actualizar el diseño, ajustar la arquitectura, desarrollar nuevos componentes, modificar código y repetir pruebas, lo que retrasó la entrega. Este ejemplo evidencia cómo un cambio tardío puede tener un impacto considerable en cascada.

9. Duración y costos

La duración de un proyecto en cascada no está predeterminada: puede ser de semanas, meses o incluso años, dependiendo del alcance, la cantidad de funcionalidades, el tamaño del equipo y los requisitos regulatorios. Por ello, no debe afirmarse que siempre dura entre seis y dieciocho meses.

El costo tampoco está definido por la metodología, sino por factores como número de desarrolladores, experiencia del equipo, tecnologías utilizadas, infraestructura, pruebas, seguridad y cambios solicitados. Lo que sí permite cascada es establecer un presupuesto

más claro cuando los requisitos son estables, aunque los costos pueden aumentar si aparecen modificaciones importantes en etapas avanzadas.

10. Cuándo usar y cuándo evitar cascada

La metodología en cascada puede ser adecuada cuando los requisitos son claros y estables, existen pocas probabilidades de cambios importantes, se necesita documentación detallada, hay requisitos regulatorios, se requiere trazabilidad y el costo de un cambio tardío es elevado.

Por el contrario, puede resultar menos conveniente cuando los requisitos no están definidos, el mercado cambia rápidamente, el cliente necesita retroalimentación continua, se requiere lanzar una versión inicial rápidamente o existe incertidumbre técnica. En estos casos, modelos iterativos o ágiles suelen ser más apropiados.

11. Evolución hacia enfoques adaptativos

Las limitaciones de cascada impulsaron el desarrollo de metodologías adaptativas como el Adaptive Software Development (ASD), que trabaja mediante ciclos de especulación, colaboración y aprendizaje. A diferencia de cascada, ASD acepta que los requisitos pueden evolucionar y que la retroalimentación continua es necesaria.

La comparación entre ambos enfoques muestra que cascada es predictiva, con planificación detallada y documentación extensa, mientras que ASD es adaptativo, con planificación ajustable, cambios esperados y colaboración frecuente con el cliente. En síntesis, cascada busca predecir y controlar, mientras que ASD busca responder al cambio y aprender durante el desarrollo.

12. Conclusión

La metodología en cascada representa un enfoque fundamental en la historia de la ingeniería de software. Su fortaleza radica en la planificación, documentación y control, lo que la hace adecuada en proyectos con requisitos estables y necesidad de trazabilidad. No obstante, su principal limitación aparece ante cambios tardíos, que pueden incrementar significativamente los costos.

Comprender sus ventajas y desventajas permite seleccionar de manera consciente el enfoque más apropiado para cada proyecto. Cascada no debe considerarse obsoleta, sino

una alternativa válida en determinados contextos, especialmente cuando se requiere previsibilidad y documentación rigurosa.

Referencias bibliográficas

- Benington, H. D. (1956). *Production of Large Computer Programs*. Symposium on Advanced Programming Methods for Digital Computers, Office of Naval Research.
- Boehm, B. W. (1981). *Software Engineering Economics*. Prentice-Hall.
- Royce, W. W. (1970). *Managing the Development of Large Software Systems*. Proceedings of IEEE WESCON.
- Garzás, J. (2014). *El verdadero origen del ciclo de cascada*.
- Brull, R. (2018). *Ingeniería del Software. Metodología Cascada*.
- Institute of Project Management. *¿Qué es la metodología en cascada? Todo lo que necesitas saber?*
- Wimi. *Método de cascada: una guía de introducción para principiantes*.
- Pipedrive. *Metodología de cascada*.
- Rock. *Metodología de cascada*.